

Verification Plan: Register File

1. DUT Overview

The DUT is a simple Register file with 32 entries, each 16 bits wide . It has1 write port and 2 read ports. Additionally it also includeserror detection for Write Read conflicts and Read Read conflicts (both reads ports same address). A reset wire reset the register to base state wth all register set to 0 and error flag also set to 0.

Signal	Direction	Width	Description
wr_en	Input	1	Write enable
wr_addr	Input	5	Write address
wr_data	Input	16	Write data
rd_addr1	Input	5	Read port 1 address
rd_addr2	Input	5	Read port 2 address
clk	Input	1	Clock signal
rst_n	Input	1	Active-low synchronous reset
rd_data1	Output	16	Read port 1 output data
rd_data2	Output	16	Read port 2 output data
err	Output	1	Error flag (registered — reflects previous cycle)

2. Features to Verify & Test Strategy per Feature

Feature	Description	Test Strategy
Reset		
Reset clears all registers	rst_n=0 → all registers read as 0 on next rising edge	Write known data to several registers, assert reset, read back all written registers and verify 0
Reset clears err	rst_n=0 → err is cleared to 0 on next rising edge	Force err=1 via illegal condition, then assert reset
Reset overrides write	If rst_n=0 and wr_en=1 simultaneously, write is suppressed	Drive wr_en=1 , wr_data=nonzero , rst_n=0 on same cycle

Write		
Synchronous write	Data appears in register on the rising edge after <code>wr_en=1</code>	Write to an address; read it back on the following cycle
Correct write address	Only the addressed register is updated	Verified by random tests via reference model comparison
Correct write data	Register stores exactly the driven value	Write constrained-random <code>wr_data</code> values to random addresses
Write enable control	No write occurs when <code>wr_en=0</code>	Drive <code>wr_en=0</code> with valid <code>wr_addr / wr_data</code> ; then read; also covered by random tests
Read		
Combinational read update	<code>rd_data</code> changes immediately when <code>rd_addr</code> changes, no clock needed	Change <code>rd_addr</code> mid-cycle; observe <code>rd_data</code> without a clock edge
Correct read address	Each port reads only its addressed register	Write distinct values to two addresses; read both simultaneously
Correct read value	Read data matches what was last written	Write then read back a range of values
Error		
<code>err</code> on rd/wr conflict	<code>err</code> asserts one cycle after <code>wr_en && (wr_addr==rd_addr1 wr_addr==rd_addr2)</code>	Drive conflict on each read port separately; follow each with a clean cycle
<code>err</code> on rd/rd conflict	<code>err</code> asserts one cycle after <code>rd_addr1==rd_addr2</code>	Drive equal read addresses, then unequal; random tests apply weighted probability of conflict
Write suppressed on error	When illegal condition is live, write does not occur on current rising edge	Trigger conflict while <code>wr_en=1</code> ; verified implicitly via reference model comparison
Read outputs X on error	<code>rd_data1</code> and <code>rd_data2</code> go to X combinationaly when illegal condition is present	Trigger conflict; verify <code>rd_data</code> goes to X combinationaly

`err` de-asserts
after conflict
resolves

`err` returns to 0 the cycle
after the illegal condition is
removed

Assert illegal condition for one
cycle, then remove it; verify
`err` goes low on following
edge

3. Testbench Architecture

- **Reference model** — behavioral `reference_RegFile` in SV; driven with identical inputs as DUT; produces expected `rd_data1`, `rd_data2`, and `err`
- **Directed generator** — runs first; covers all features listed in Section 2
- **Constrained-random generator** — starts after directed phase completes (`dir_done`); runs 1000 transactions with:
 - 5% `rst_n=0`, 95% `rst_n=1`
 - 80% `wr_en=1`, 20% `wr_en=0`
 - 3% all-zeros write data, 2% all-ones, 95% uniform random
 - 10% forced `rd_addr1 == rd_addr2` (rd/rd conflict), 90% free
- **Driver** — muxes directed/random stimulus onto DUT and reference model inputs; switches to random on `dir_done`
- **Monitor** — combinational pass-through of all DUT and reference signals; provides a `valid` flag (registered, high after first posedge) to gate scoreboard checks during initialization
- **Scoreboard** — performs two levels of checking per cycle:
 - i. **Combinational read check** — between clock edges check whenever a read address changes, compares DUT `rd_data` against the reference 1 ns after clock cycle start. This catches DUTs which reads are not combinational
 - ii. **Clocked check** — clocked check on each posedge: compares `rd_data1`, `rd_data2`, `err` between DUT and reference;

4. Coverage Goals

Directed phase — each scenario exercised at least once (not on all registers) - total 10 test:

- All 4 error types triggered: rd/rd conflict, wr/rd port 1, wr/rd port 2, err cleared by reset
- Reset verified on at least 4 distinct registers
- Both read ports covered independently

Randomized phase — 1000 transactions:

- Write addresses spread across the full range (0–31)
- `wr_data=0x0000` and `wr_data=0xFFFF` each hit
- rd/rd conflict hit ~100 times
- wr/rd conflict hit 33+ times (99%+ probability of exceeding 33)
- `wr_en=0` cycles ~200 times — verifies no phantom writes

Not covered exhaustively:

- not all 32 registers are written and read back in directed tests
- no cross-coverage between error type × register address × data value × write enable

5. Bug Table

Design	Bugs
<code>regfile_v0</code>	None — passes all tests
<code>regfile_v1</code>	rd/rd conflict not detected: no X on read ports, <code>err</code> stays 0
<code>regfile_v2</code>	Error handling delayed by one cycle (except <code>err</code> flag): X outputs appear one cycle late (on both rd/rd and wr/rd conflicts), and wr/rd conflict blocks the write on the next cycle instead of the current one
<code>regfile_v3</code>	wr/rd conflict does not suppress the write: register is updated despite the conflict

Other

Total amount of tests: 10 direct + 1 randommized

10 directed to go through the specific cases at least once and visually easy to detect errors. This is generated by the `directed_generator`.

Then 1 randomized test with 1000 transactions to expend the coverage and test more scenerios registers etc. then the directed. Generated by the `random_generator`.

Time to run :

As a clock cycle is around 10ns and total cyles are made up of the 32 cycles for the directed test and the 1000 cycles of the constraint randomized test. Therefore the total run time is around ~10 320 ns.

AI Usage

- Report formatting/ wording
- print statements
- syntax help
- code clean up
- coding help (e.g. move variables to top, wire initialization in the final tb_regfile module - mostly to avoid repetitive writing tasks)

Time spend on assignment

Around 5-10 hours (excluding lab)

Note: I often work/code on the side of doing something else so its hard to track how much time I actually spent on coding tasks etc.

Clocking Blocks

Clocking blocks allow for specifying the timing of signals repsective to their clock. They allow for delaying the chnage on input and output wires. This can help with avoiding race conditions aswell as sampleing after clock edges. Often used in interfaces they can help with organizing and simplify interface timings and allow for a centralized overview of the tb timings.

In my test bench instead of using a clocking block to prevent race condition between the refrence models and presumable DUT `always_ff @(posedge clk)` and the send task from my driver, I used `@(posedge clk); #1;` to delay the changes on the wires by 1ns.

Alternativly I could have included a Clocking block in the driver to delay its outputs by 1ns to achive the same result. As this is a quite small testbench this approach works, however if the code gets larger and a bit more complex using proper clocking blocks will likly be a better approach to avoid reptative timing managment which can likly (I imagine) lead to clocking errors hard to find and debug.